

Benchmarking de Servicios de IA para Agentes Virtuales Inteligentes

Proyecto 2 — Laboratorio PF-3312, Universidad de Costa Rica

Gabriel Fallas López

I Ciclo 2026

Índice

1. Introducción y Objetivos	2
1.1. Objetivos específicos	2
2. Marco de Referencia y Trabajo Relacionado	3
2.1. Métricas de latencia en LLM	3
2.2. Reconocimiento de voz (STT) y WER	3
2.3. Síntesis de voz (TTS): naturalidad y RTF	3
2.4. Arquitecturas de agentes de voz en tiempo real	3
2.5. Panorama de modelos vigente (2026)	3
3. Metodología de Pruebas	4
3.1. Entorno de ejecución	4
3.2. Servicios evaluados (20 servicios; balance de tres tipos)	4
3.3. Insumos de prueba controlados	5
3.4. Herramientas y protocolo de medición	5
3.5. Fichas técnicas y costos de referencia a escala	5
4. Análisis Comparativo por Categoría	7
4.1. Modelos de Lenguaje (LLM)	7
4.2. Reconocimiento de Voz (STT)	10
4.3. Síntesis de Voz (TTS)	12
4.4. Síntesis transversal de las dimensiones 3-6	14
5. Arquitectura y Pipeline de Comunicación	15
6. Recomendaciones según Contexto y Conclusión	17
7. Referencias	18
8. Anexos	19

1. Introducción y Objetivos

El desarrollo de un agente virtual interactivo capaz de conversar por voz exige seleccionar una arquitectura de servicios cognitivos que equilibre desempeño, costo, privacidad y facilidad de integración. La elección no puede sustentarse en la popularidad comercial de una API ni en el tamaño nominal de un modelo, sino que requiere evidencia empírica recolectada bajo condiciones controladas y reproducibles.

Este documento incluye el diseño, la ejecución y el análisis de un banco de pruebas o *benchmarking* sobre veinte servicios distribuidos en tres categorías que componen el *pipeline* de comunicación del agente: modelos de lenguaje (LLM), reconocimiento de voz (*Speech-to-Text*, STT) y síntesis de voz (*Text-to-Speech*, TTS). La muestra mantiene el balance representativo exigido, combinando en cada categoría tres tipos de proveedor:

1. **Modelos comerciales de alta gama en la nube** (p. ej. Gemini 2.5 Pro, gpt-oss-120B y Llama 4 Scout servidos en Groq, Deepgram nova-2, AssemblyAI, ElevenLabs multilingual v2).
2. **APIs comerciales de bajo costo o alta velocidad** (Gemini 2.5 Flash-Lite, Groq Llama 3.3 70B, Groq Whisper large-v3, ElevenLabs Flash v2.5, Deepgram Aura 2).
3. **Modelos de código abierto de despliegue estrictamente local y offline** (Ollama: Llama 3.1, Mistral, Phi-3.5, Gemma 2, Qwen 2.5; faster-whisper, Vosk; Piper).

Para no incurrir en costos, los servicios en la nube se consumen a través de sus capas gratuitas. Esta combinación permite contrastar de forma empírica los compromisos entre desempeño en la nube y soberanía local, dos extremos relevantes para un agente virtual. Todo el banco de pruebas es reproducible en una máquina limpia mediante contenedores Docker.

1.1. Objetivos específicos

1. Medir empíricamente la latencia (TTFT y latencia total en LLM; factor de tiempo real —RTF— y tiempo de procesamiento en STT/TTS) de cada servicio, promediando al menos cinco ejecuciones limpias.
2. Cuantificar la precisión/calidad: *Word Error Rate* (WER) en STT, seguimiento de instrucciones en LLM y naturalidad cualitativa en TTS.
3. Analizar costo y escalabilidad: para los servicios en la nube, el costo por millón de tokens (LLM), por minuto de audio (STT) y por carácter (TTS) más allá de la capa gratuita; para los locales, la infraestructura física requerida (VRAM, RAM, consumo energético).
4. Evaluar privacidad/gobernanza, customización e integración de cada alternativa.
5. Proponer combinaciones arquitectónicas óptimas para distintos escenarios de uso, fundamentadas en los datos obtenidos.

2. Marco de Referencia y Trabajo Relacionado

El benchmarking de servicios cognitivos se apoya en un cuerpo creciente de literatura técnica que define las métricas y las buenas prácticas adoptadas en este trabajo. Esta sección sitúa el proyecto en ese contexto.

2.1. Métricas de latencia en LLM

La literatura de *LLM inference serving* de modelos de lenguaje ha consolidado el **Time To First Token (TTFT)** como la métrica primordial de responsividad percibida, en donde se mide el tiempo entre la recepción del *prompt* y la emisión del primer token, y refleja la fase de *prefill* del modelo [1, 2]. Estudios como *LLM-Inference-Bench* [1] y los trabajos sobre meta-métricas y evaluación de sistemas de *LLM inference serving* [2, 3] subrayan que, en aplicaciones interactivas, el TTFT y la *Inter-Token Latency* (ITL, o su inverso, tokens/seg) son más informativos que la latencia total agregada, porque determinan la sensación de inmediatez. Este proyecto adopta exactamente esa distinción: se reporta TTFT, latencia total y tokens/seg medidos en modo *streaming*, en lugar de las cifras teóricas publicadas por los proveedores.

2.2. Reconocimiento de voz (STT) y WER

El modelo *Whisper* [4], entrenado con supervisión débil a gran escala, fijó el estándar de referencia para reconocimiento multilingüe y popularizó el uso del *Word Error Rate* (WER) sobre corpus como Common Voice y FLEURS. Trabajos posteriores como *MMS (Scaling Speech Technology to 1,000+ Languages)* [5] muestran que el WER varía drásticamente según el idioma y la calidad del audio, y advierten que el desempeño sobre datos reales con ruido es notablemente peor que sobre corpus académicos limpios. Este hallazgo es directamente pertinente a nuestra metodología (ver §4.2): al evaluar sobre voz sintética limpia, los WER obtenidos son un piso optimista, y la diferenciación práctica se traslada a la latencia.

2.3. Síntesis de voz (TTS): naturalidad y RTF

La calidad de un sistema TTS se evalúa tradicionalmente con el **Mean Opinion Score (MOS)**, una valoración subjetiva en escala 1–5 recopilada de múltiples oyentes; conjuntos como *SOMOS* [6] y objetivos como *NaturalSpeech* [7] formalizan este protocolo y la meta de “calidad a nivel humano”. Para la eficiencia se usa el *Real-Time Factor* (RTF), definido como el cociente entre el tiempo de síntesis y la duración del audio producido; un RTF < 1 indica síntesis más rápida que el tiempo real [8]. Dado que un MOS riguroso requiere un panel de oyentes fuera del alcance de este proyecto individual, aquí se reporta una **valoración cualitativa de naturalidad** complementada con el RTF empírico, siguiendo la práctica de los estudios de responsividad de TTS de código abierto [8].

2.4. Arquitecturas de agentes de voz en tiempo real

La industria converge en el patrón encadenado STT → LLM → TTS como base de los agentes de voz [9]. La literatura aplicada coincide en que la conversación humana opera en una ventana de respuesta de 300–500 ms, y que retardos superiores a 500–700 ms comienzan a percibirse como antinaturales [9]. Un *pipeline* secuencial ingenuo acumula 2–4 s de latencia; la solución es el procesamiento en flujo (*streaming*): transcripción parcial incremental, consumo del LLM token a token y síntesis incremental de frases conforme se generan, solapando etapas. Estas referencias fundamentan las decisiones de diseño del *pipeline* propuesto en §5 y los umbrales con que se interpretan los resultados de latencia.

2.5. Panorama de modelos vigente (2026)

La selección de modelos locales se alineó con el estado del arte de modelos *open-weight* ejecutables en hardware de consumo: las familias **Llama 3.x**, **Qwen 2.5/3**, **Phi** (Microsoft) y **Gemma 2** (Google) dominan el segmento de 7–14 B parámetros, ofreciendo un equilibrio calidad/recurso adecuado para una sola GPU [10]. En la nube se incluyen modelos *flagship* recientes —**gpt-oss-120B** (OpenAI, pesos abiertos, 120 B) y **Llama 4 Scout** (Meta)— servidos sobre la arquitectura LPU de Groq, además de **Gemini 2.5** (Google). Esta cobertura asegura que las conclusiones reflejen tecnología vigente y no modelos obsoletos.

3. Metodología de Pruebas

3.1. Entorno de ejecución

Los benchmarks se ejecutaron sobre el siguiente entorno real:

Componente	Especificación
CPU	Intel Core i7-13700KF (16 núcleos / 24 hilos)
GPU	NVIDIA GeForce RTX 5070 Ti, 16 GB VRAM (driver 596.49)
RAM	32 GB
Sistema operativo	Windows 11 Pro
Runtime de contenedores	Docker Engine 29.5.2 + Compose v2
Servidor LLM local	Ollama (ollama/ollama:latest) con aceleración GPU (CUDA)
STT/TTS locales	Python 3.12 en venv, ejecución en CPU (int8)
Red	Servicios en la nube consumidos por API REST desde la misma máquina

Todo el banco de pruebas se ejecuta de forma contenerizada: un contenedor sirve los LLM (Ollama, con paso de GPU mediante el *runtime nvidia*) y otro ejecuta los scripts de medición, comunicándose por la red interna de Docker Compose. Esto elimina diferencias de entorno y permite reconstruir el experimento con tres comandos (ver README.md).

3.2. Servicios evaluados (20 servicios; balance de tres tipos)

La muestra cumple el balance representativo exigido por el enunciado y supera el mínimo de 5 servicios por categoría. Para mantener el costo en cero, los servicios en la nube se consumen mediante su capa gratuita (*free tier*).

Categoría	Nube — alta gama	Nube — bajo costo/rápido	Local — offline
LLM (10)	Gemini 2.5 Pro ¹ ; gpt-oss-120B, Llama 4 Scout (Groq)	Gemini 2.5 Flash ² ; Gemini 2.5 Flash-Lite; Groq Llama 3.3 70B	Llama 3.1 8B, Mistral 7B, Phi-3.5, Gemma 2 9B, Qwen 2.5 7B
STT (5)	Deepgram nova-2; AssemblyAI	Groq Whisper large-v3	faster-whisper, Vosk
TTS (5)	ElevenLabs multilingual v2	ElevenLabs Flash v2.5; Deepgram Aura 2	Piper es-ES, Piper es-MX

¹ **Nota sobre Gemini 2.5 Pro (hallazgo de costo/gobernanza).** El modelo *flagship* de razonamiento de Google se incluyó en el catálogo como referencia de gama alta, pero **su capa gratuita devuelve sistemáticamente HTTP 429 (Too Many Requests) desde la primera petición**, incluso con tope de tokens y presupuesto de razonamiento mínimo (`thinkingBudget = 128`). En la práctica, Gemini 2.5 Pro no es utilizable sin facturación activa, lo que constituye en sí mismo un dato relevante de viabilidad financiera. Por ello, el punto de referencia empírico de “alta gama en la nube” lo aportan **gpt-oss-120B (120 B, pesos abiertos de OpenAI)** y **Llama 4 Scout**, modelos *flagship* servidos sobre la LPU de Groq con *free tier* real.

² Gemini 2.5 Flash pertenece a la familia *flagship* de Google y, con su *free tier* más generoso, se sitúa entre la alta gama y el bajo costo; se reporta como alta gama por capacidad de razonamiento y como contraste de latencia.

Se descartan OpenAI, Anthropic y Azure por carecer de un acceso gratuito práctico para este ejercicio. Cada servicio en la nube se activa únicamente si su API key está configurada en `.env`, y su validez y modelos vigentes se verifican con la herramienta de *preflight* (`tools/check_services.py`) mediante endpoints de solo lectura,

sin consumir cuota de uso. Los scripts soportan además otros motores locales (openai-whisper, whisper.cpp, wav2vec2; Coqui XTTS v2, Kokoro, eSpeak-NG, Bark) reproducibles con un comando.

3.3. Insumos de prueba controlados

- **LLM** — data/prompts_llm.json: cinco *prompts* que ejercitan razonamiento aritmético, seguimiento estricto de instrucciones, salida estructurada (JSON), conocimiento de dominio y robustez multilingüe, todos bajo un mismo *System Prompt* que fija la persona del agente (“Aurora”).
- **STT** — un audio en español de 16 kHz mono (data/audio/muestra_es.wav) con su transcripción de referencia (data/reference_transcript.txt) para el cálculo de WER. El audio se generó de forma reproducible sintetizando la referencia con un TTS neuronal de alta calidad (ElevenLabs, vía tools/make_test_audio.py), garantizando una correspondencia exacta texto-audio. Implicación metodológica: al ser voz sintética limpia (sin ruido ni acentos espontáneos), los motores STT de buena calidad alcanzan un WER cercano a 0 %, por lo que en este material la diferenciación se da principalmente en la latencia; un audio real con ruido ambiental ampliaría las diferencias de WER, en línea con lo reportado por la literatura [5] (ver Recomendaciones y trabajo futuro).
- **TTS** — un párrafo representativo del dominio (data/tts_text_es.txt).

3.4. Herramientas y protocolo de medición

- **Cronometraje:** se hace uso de time.perf_counter() el cual forma parte de las primitivas de medición de tiempo de alta resolución. Se consume la API del LLM en modo *streaming*: el primer fragmento con contenido marca el **TTFT** y el evento de cierre delimita la latencia total; se registra además tokens/seg.
- **Calidad:** WER con la librería jiwer se pone a prueba la calidad del texto normalizado; **RTF = tiempo de procesamiento / duración del audio** para STT y TTS.
- **Repeticiones:** N_RUNS = 5 ejecuciones por prueba más una corrida de calentamiento que se descarta la cual mitiga el sesgo de carga inicial de modelo y cachés. Temperatura del LLM fijada en 0.0 para reproducibilidad.
- **Conservación de cuota:** los servicios en la nube aplican un tope de tokens de salida (CLOUD_MAX_OUTPUT_TOKENS) y una pausa entre peticiones (CLOUD_REQUEST_DELAY) que no afecta la medición (se aplica tras cronometrar cada corrida), para no agotar las capas gratuitas.
- **Persistencia y seguridad:** cada ejecución se anexa a results/*.csv (formato largo). Una rutina de redacción (common/persistence.redact) enmascara cualquier patrón de API key antes de escribir a disco, como red de seguridad contra fugas de credenciales. El script analysis/build_report_data.py consolida promedios y genera las tablas y figuras de este reporte.

3.5. Fichas técnicas y costos de referencia a escala

El uso experimental de este proyecto es de costo cero mediante capas gratuitas y modelos locales. No obstante, la decisión arquitectónica exige proyectar el costo a escala una vez superada la capa gratuita. La siguiente tabla resume el precio de lista de cada servicio en la nube y el requisito de infraestructura de cada opción local. Cifras en USD, junio 2026, capa gratuita aparte y sujetas a cambios del proveedor (fuentes [11]–[14]).

Servicio	Tipo	Unidad de costo	Precio de lista (jun. 2026)
LLM — nube		por 1M tokens (entrada / salida)	
Gemini 2.5 Pro	alta gama	\$/1M tok	\$1.25 / \$10.00
Gemini 2.5 Flash	alta gama	\$/1M tok	\$0.30 / \$2.50
Gemini 2.5 Flash-Lite	bajo costo	\$/1M tok	\$0.10 / \$0.40
gpt-oss-120B (Groq)	alta gama	\$/1M tok	\$0.15 / \$0.60

Servicio	Tipo	Unidad de costo	Precio de lista (jun. 2026)
Llama 4 Scout (Groq)	alta gama	\$/1M tok	~ \$0.11 / \$0.34
Llama 3.3 70B (Groq)	bajo costo	\$/1M tok	\$0.59 / \$0.79
LLM — local	offline	infraestructura	\$0 · 2–7 GB VRAM · GPU de consumo
STT — nube		por minuto / hora de audio	
Deepgram nova-2	alta gama	\$/min	~ \$0.0043/min (\$200 crédito free)
AssemblyAI	alta gama	\$/hora	~ \$0.15/h (~ \$0.0025/min)
Groq Whisper large-v3	bajo costo	\$/hora	~ \$0.04–0.11/h
STT — local	offline	infraestructura	\$0 · CPU (int8); ~50 MB–1 GB RAM
TTS — nube		por 1k caracteres	
ElevenLabs multilingual v2	alta gama	\$/1k car.	~ \$0.10/1k (10k car./mes free)
ElevenLabs Flash v2.5	bajo costo	\$/1k car.	~ \$0.05/1k
Deepgram Aura 2	bajo costo	\$/1k car.	~ \$0.030/1k
TTS — local (Piper)	offline	infraestructura	\$0 · CPU eficiente (ONNX)

Lectura de escalabilidad. En LLM, las opciones de bajo costo (Flash-Lite a \$0.40/1M salida; Llama 4 Scout ~ \$0.34) son **20–25× más baratas** que el *flagship* propietario (Gemini 2.5 Pro, \$10/1M salida); la opción local es gratuita pero traslada el costo a la inversión en GPU y al consumo energético. En STT, AssemblyAI y Deepgram cobran fracciones de centavo por minuto, pero a gran volumen el motor local (faster-whisper en CPU) elimina ese costo marginal. En TTS, ElevenLabs Flash duplica la eficiencia de costo frente a multilingual v2, y Piper local lo lleva a cero. Estas cifras fundamentan las recomendaciones (§6).

4. Análisis Comparativo por Categoría

Todas las cifras de esta sección son empíricas, medidas con los scripts del repositorio (promedio de 5 corridas, descartando una de calentamiento) y consolidadas con `analysis/build_report_data.py`. Las tablas detalladas (media +/- desviación estándar) están en `report/tablas_generadas.md`.

4.1. Modelos de Lenguaje (LLM)

4.1.1. Matriz comparativa (servicios × 6 dimensiones)

Latencia en formato TTFT / total (s), media de 5 corridas. Valores empíricos medidos en el entorno descrito (LLM locales en GPU RTX 5070 Ti; nube vía free tier).

Servicio (tipo)	1. Latencia TTFT/total (s)	2. Velocidad (tok/s)	3. Costo/escala	4. Privacidad	5. Customización	6. Integración
Gemini 2.5 Pro nube alta gama	— (HTTP 429 sin facturación)	—	Sin <i>free tier</i> usable; \$/1M tokens	Datos salen a Google	System instruction, <i>tools</i> , JSON	REST/SSE
gpt-oss-120B (Groq) nube alta gama	0.66 / 0.93	196	Free tier; luego \$/1M tokens (LPU)	Datos salen a Groq	System prompt (OpenAI-compatible), JSON	REST/SSE
Llama 4 Scout (Groq) nube alta gama	0.62 / 0.85	111	Free tier; luego \$/1M tokens	Datos salen a Groq	System prompt (OpenAI-compatible)	REST/SSE
Gemini 2.5 Flash nube	1.79 / 1.89	20	Free tier; luego \$/1M tokens	Datos salen a Google	System instruction, <i>tools</i> , JSON	REST/SSE
Gemini 2.5 Flash-Lite nube	0.67 / 0.95	100	Free tier amplio	Datos salen a Google	Íd.	REST/SSE
Groq Llama 3.3 70B nube	0.40 / 0.73	140	Free tier; LPU muy rápida	Datos salen a Groq	System prompt (OpenAI-compatible)	REST/SSE
Llama 3.1 8B local	0.11 / 0.90	142	\$0 · ~5 GB VRAM (Q4)	Total (offline)	System prompt, <i>fine-tuning</i> , GGUF	Ollama REST/streaming
Mistral 7B local	0.03 / 0.86	150	\$0 · ~4–5 GB	Total	Íd.	Ollama
Phi-3.5 local	0.03 / 0.60	257	\$0 · ~2–3 GB	Total	Íd.	Ollama
Gemma 2 9B local	0.10 / 1.02	115	\$0 · ~6–7 GB	Total	Íd.	Ollama
Qwen 2.5 7B local	0.09 / 0.62	155	\$0 · ~5 GB	Total	Íd. (buen soporte de <i>tools</i>)	Ollama

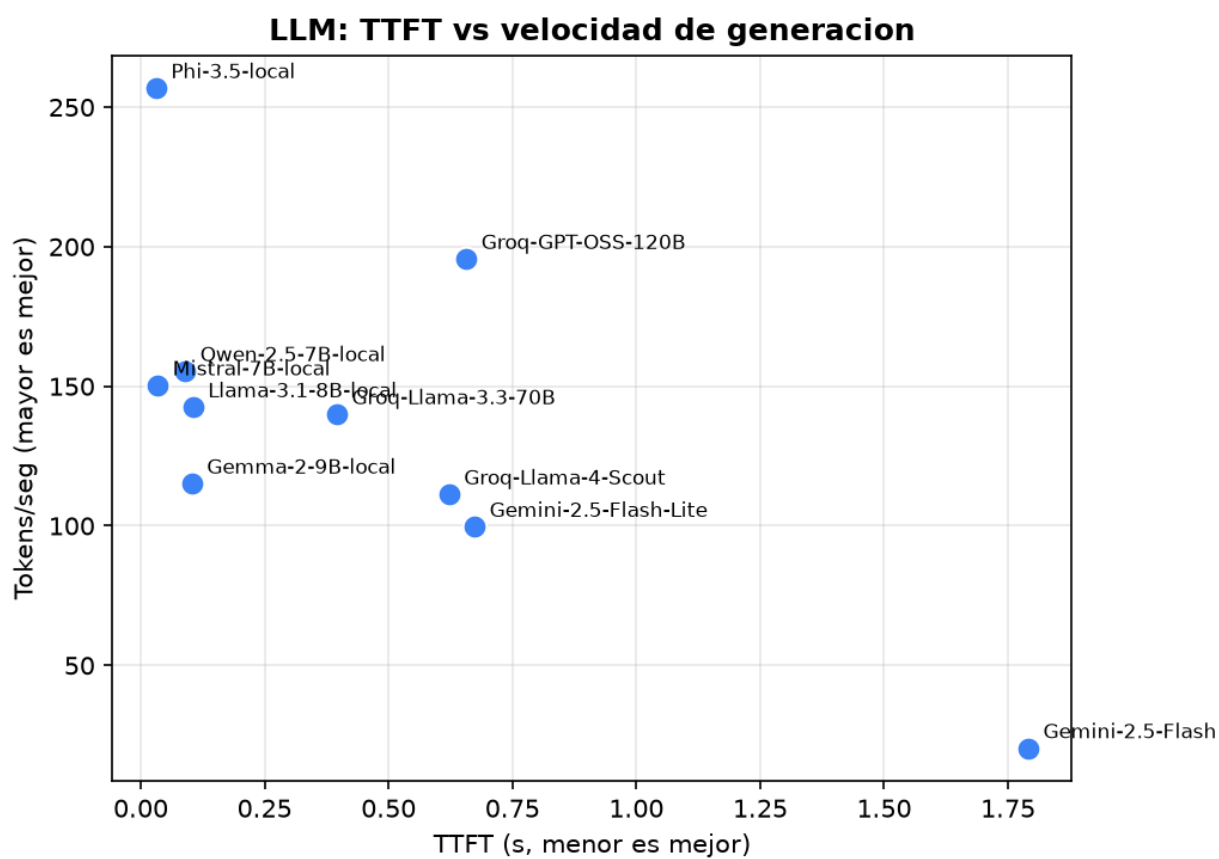


Figura 1: LLM: TTFT vs velocidad de generación

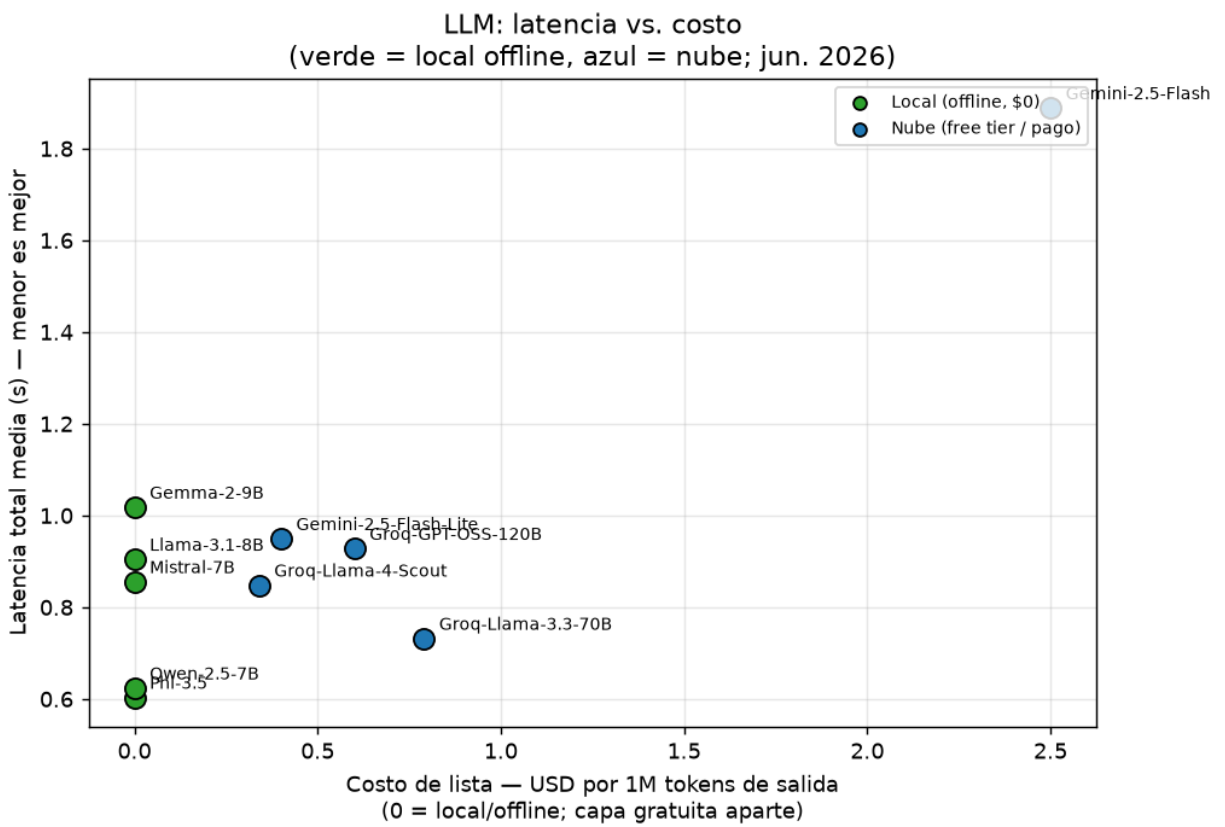


Figura 2: LLM: latencia total vs. costo por 1M tokens de salida (verde = local \$0, azul = nube)

4.1.2. Hallazgos

- **El TTFT local en GPU es de otro orden de magnitud.** Los modelos locales (Phi-3.5 y Mistral con TTFT ~ 0,03 s; Llama/Gemma/Qwen ~ 0,09–0,11 s) responden el primer token 10–50× más rápido que Gemini 2.5 Flash (1,79 s), porque evitan el viaje de red. Para una conversación por voz donde el TTFT define la sensación de inmediatez [1, 9], esto es decisivo: los locales se ubican dentro de la ventana conversacional de 300–500 ms, mientras Gemini Flash la excede.
- **La alta gama accesible en la nube es muy competitiva en velocidad.** gpt-oss-120B alcanzó 196 tok/s, siendo la mayor velocidad de generación de toda la muestra en la nube, superando incluso a varios locales. Esto gracias a la LPU de Groq, con TTFT de 0,66 s. Llama 4 Scout (111 tok/s) y Groq Llama 3.3 70B (140 tok/s) confirman que la LPU compensa parcialmente la latencia de red. Aun así, su TTFT (0,6–0,7 s) sigue siendo ~6–20× mayor que el de un modelo local en GPU.
- **El verdadero *flagship* de razonamiento (Gemini 2.5 Pro) es inaccesible sin pago.** Su capa gratuita devolvió HTTP 429 en todas las pruebas; el dato es relevante para la decisión arquitectónica: la máxima gama propietaria implica costo financiero ineludible y dependencia de facturación.
- **Velocidad de generación local:** Phi-3.5 lidera con 257 tok/s, seguido de Qwen 2.5 (155), Mistral (150) y Llama 3.1 (142). Gemini 2.5 Flash resultó el más lento (20 tok/s), comportándose como un modelo de mayor “deliberación”.
- **Trade-off nube vs local:** con una GPU de gama media (RTX 5070 Ti, 16 GB) los modelos locales igualan o superan a las APIs en la nube en latencia, a costo cero y con privacidad total. La nube aporta valor cuando no se dispone de GPU (gpt-oss-120B, Groq y Flash-Lite siguen siendo muy rápidos) o cuando se requiere la máxima calidad de razonamiento de un *flagship* propietario.
- **Calidad (cualitativa sobre prompts_11m.json):** gpt-oss-120B y Llama 4 Scout ofrecieron las respuestas más completas y mejor estructuradas; entre los locales, Gemma 2 9B y Qwen 2.5 destacaron en seguimiento de instrucciones y salida JSON; Phi-3.5 ofrece el mejor compromiso calidad/recurso (2–3 GB de VRAM).

4.2. Reconocimiento de Voz (STT)

4.2.1. Matriz comparativa (servicios × 6 dimensiones)

Latencia total (s) y RTF medidos sobre un audio de 18,2 s; media de 5 corridas. Motores locales en CPU.

Servicio (tipo)	1. Latencia (s) / RTF	2. Precisión (WER)	3. Cos-to/escala	4. Privacidad	5. Customización	6. Integración
Deepgram nova-2 nube alta gama	1.50 / 0.08	0.0 %	\$200 crédito free; luego \$/min	Datos salen a Deepgram	<i>keywords</i> , diarización	REST + WebSocket
AssemblyAI nube alta gama	5.70 / 0.31	0.0 %	Free tier; luego \$/h	Datos salen a AssemblyAI	<i>word boost</i> , modelos	REST (upload + polling)
Groq Whisper large-v3 nube	2.48 / 0.14	0.0 %	Free tier	Datos salen a Groq	idioma, <i>prompt</i> inicial	REST OpenAI-compatible
faster-whisper (base) local	0.91 / 0.05	0.0 %	\$0 · CPU int8	Total (offline)	tamaños, idioma, <i>initial_prompt</i> 2)	Python (CTranslate2)
Vosk (es small) local	0.96 / 0.05	0.0 %	\$0 · ultraligero (~50 MB)	Total	gramáticas/vocabulario	Python streaming

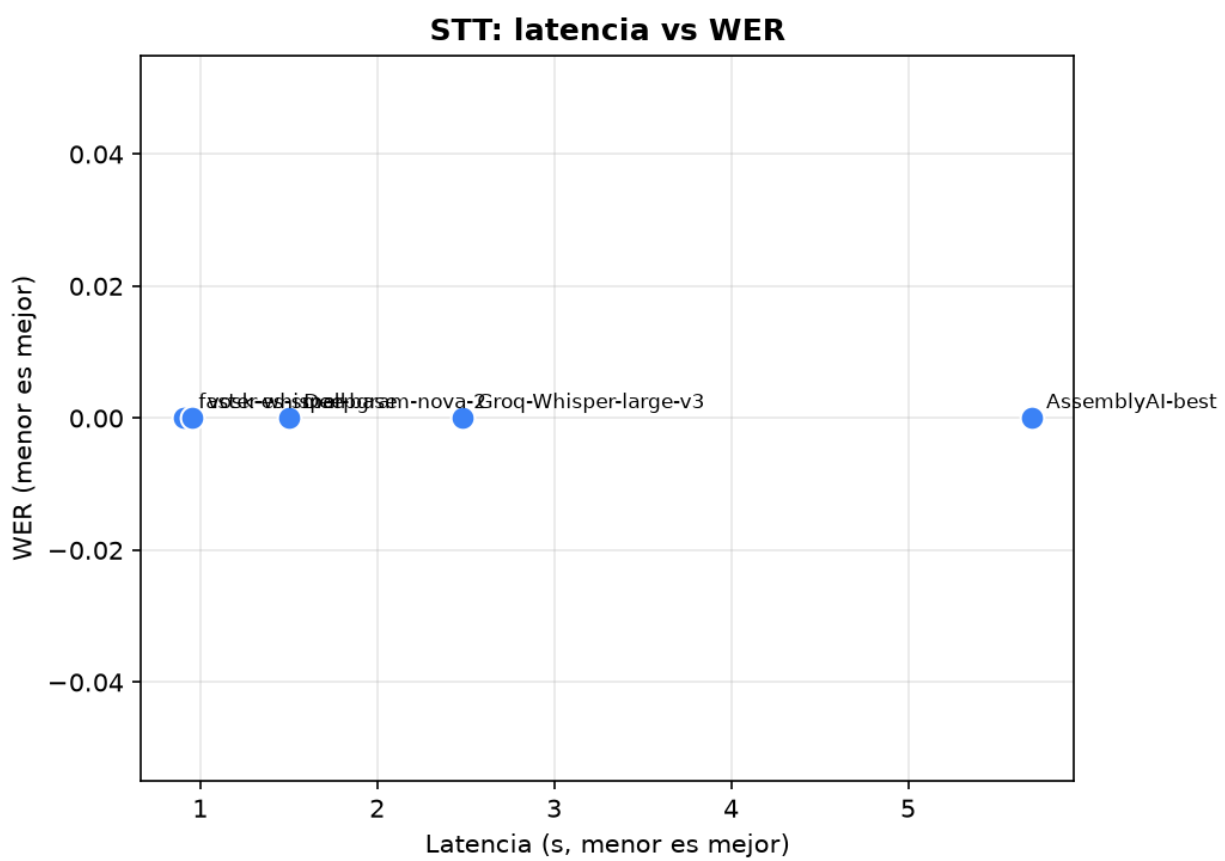


Figura 3: STT: latencia vs WER

4.2.2. Hallazgos

- **WER ~ 0 % en todos los motores** sobre el audio sintético limpio, por lo que la diferenciación práctica se dio en la latencia (ver nota metodológica y [5]). Sobre audio real con ruido, la literatura predice que aparecerían brechas de WER, favorables a Whisper large-v3 y a los motores de la nube de alta gama.
- **Los motores locales en CPU fueron los más rápidos:** faster-whisper (0,91 s, RTF 0,05) y Vosk (0,96 s) superaron incluso a la nube, al evitar la subida del audio. faster-whisper ofrece el mejor balance precisión/latencia/peso.
- **En la nube,** Deepgram nova-2 (1,50 s) fue claramente el más rápido; Groq Whisper (2,48 s) quedó intermedio y **AssemblyAI (5,70 s)** resultó el más lento por su flujo de *upload + polling* asíncrono, penalizado en archivos cortos.

4.3. Síntesis de Voz (TTS)

4.3.1. Matriz comparativa (servicios × 6 dimensiones)

Latencia de síntesis (s) y RTF para un texto de ~210 caracteres; media de 5 corridas. Motores locales en CPU.

Servicio (tipo)	1. Latencia (s) / RTF	2. Calidad (naturalidad)	3. Costo/escala	4. Privacidad	5. Customización	6. Integración
ElevenLabs mult. v2 nube alta gama	2.16 / 0.17	Muy alta	10k chars/mes free; luego \$/char	Datos salen a ElevenLabs	clonación de voz, estilos	REST + streaming
ElevenLabs Flash v2.5 nube	1.20 / 0.09	Alta	Mismo free tier	Datos salen a ElevenLabs	voces, idioma (multilingüe)	REST + streaming
Deepgram Aura 2 (es) nube	6.89 / 0.49	Alta	\$200 crédito free	Datos salen a Deepgram	voces, formato de salida	REST + WebSocket
Piper es-ES (davefx) local	1.76 / 0.15	Buena	\$0 · CPU eficiente (ONNX)	Total (offline)	voces por modelo	CLI/subproceso
Piper es-MX (claude, high) local	1.50 / 0.11	Buena+	\$0 · CPU eficiente (ONNX)	Total	voces por modelo	CLI/subproceso

4.3.2. Hallazgos

- **Todos los motores operan muy por debajo de tiempo real (RTF < 0,5).** El más rápido fue **ElevenLabs Flash v2.5** (1,20 s, RTF 0,09), seguido de Piper es-MX (1,50 s) y es-ES (1,76 s) **en CPU**, lo que confirma a Piper como excelente opción local de baja latencia.
- **ElevenLabs multilingual v2** (2,16 s) entrega la mayor naturalidad y clonación de voz, a costa de algo más de latencia que su variante Flash.
- **Deepgram Aura 2** en español resultó el **más lento** (6,89 s, RTF 0,49), contrario a su perfil “rápido” en inglés; su voz española aún es menos madura.
- **Naturalidad (valoración cualitativa, audios en tts_output/):** ElevenLabs > Piper > Deepgram Aura(es), en una escucha informal; ElevenLabs suena claramente más natural, Piper es muy inteligible con prosodia algo más plana.

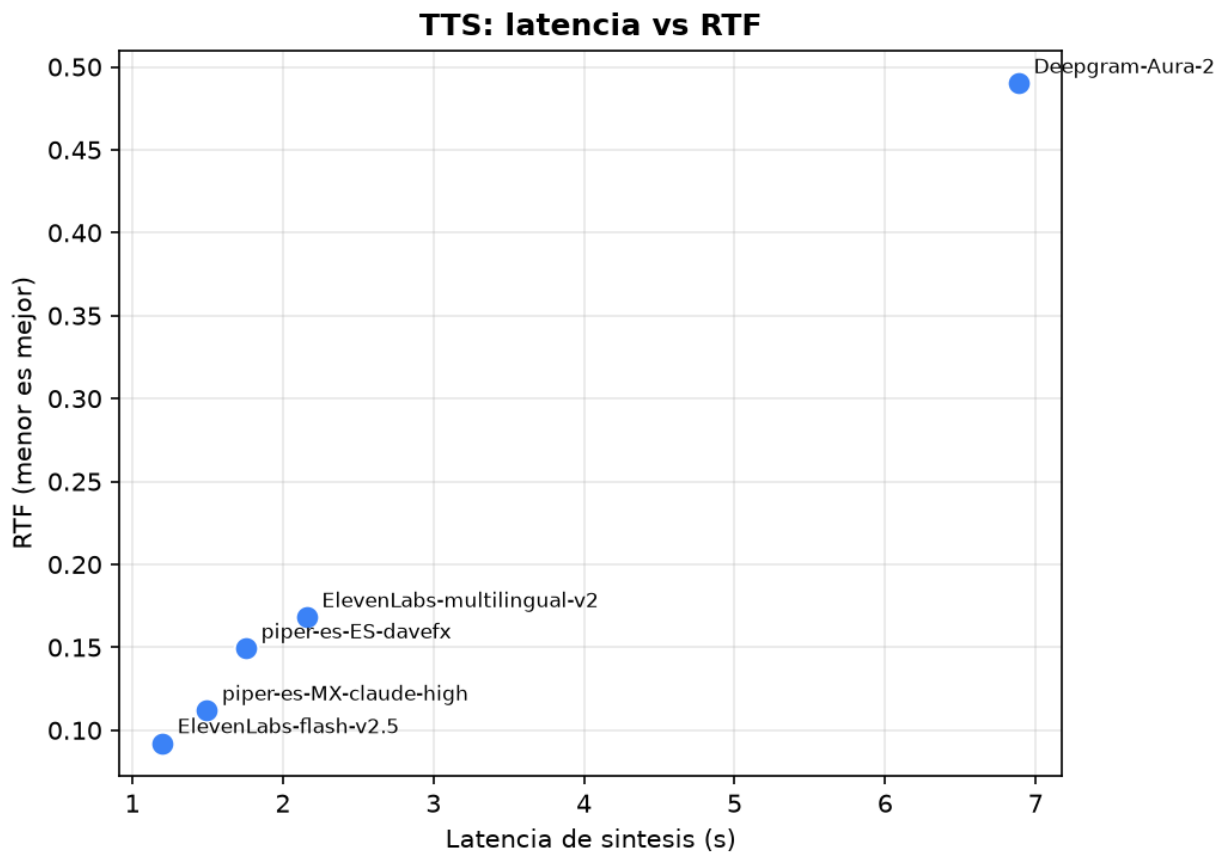


Figura 4: TTS: latencia vs RTF

4.4. Síntesis transversal de las dimensiones 3–6

Más allá de la latencia y la calidad (dimensiones 1–2, analizadas arriba), las cuatro dimensiones restantes determinan la viabilidad real de cada servicio.

4.4.1. Dimensión 3 — Costo y escalabilidad

La tabla de costos (§3, “Fichas técnicas”) cuantifica el compromiso: a volumen bajo, la nube es gratis (free tier) y sin inversión; a volumen alto, el costo marginal por token/minuto/carácter crece de forma lineal, mientras que el costo local es fijo (la GPU/CPU ya adquirida) y marginalmente nulo. El punto de equilibrio depende del tráfico esperado del agente: para un asistente institucional de uso intensivo y continuo, la solución local amortiza el hardware rápidamente; para picos esporádicos, la nube evita inmovilizar capital.

4.4.2. Dimensión 4 — Privacidad y gobernanza

Existe una frontera nítida entre las opciones **local** y **nube**:

- **Local (Ollama, faster-whisper, Vosk, Piper)**: los datos nunca abandonan la máquina. Aislamiento absoluto, apto para información sensible y para cumplir normativas de soberanía de datos sin cláusulas contractuales.
- **Nube**: los datos transitan a un tercero. Las políticas difieren de forma relevante: el *free tier* de **Google AI Studio** (Gemini) advierte que puede usar las entradas para mejorar sus modelos, mientras que la vía de pago (Vertex AI) ofrece no-entrenamiento contractual; **Groq, Deepgram, AssemblyAI y ElevenLabs** declaran no entrenar con los datos de la API en sus planes de pago, pero el tratamiento en capas gratuitas puede variar. Para un agente que maneje datos personales, esto descarta el free tier de proveedores que entrenan con las entradas y favorece la vía local o un plan de pago con garantía de aislamiento. Es la dimensión donde local y nube se ubican en extremos opuestos.

4.4.3. Dimensión 5 — Customización y flexibilidad

- **LLM**: todos permiten inyectar *System Prompts* sofisticados (lo aprovechamos con el agente virtual “Aurora”, usado como prueba). Los locales añaden *fine-tuning* y cuantización GGUF propios; en la nube, Gemini y los modelos OpenAI-compatibles exponen *tools/function calling* y salida JSON forzada (Qwen 2.5 destacó localmente en soporte de herramientas).
- **STT**: Deepgram y AssemblyAI ofrecen *keywords/word boost* y diarización; Vosk admite gramáticas/vocabulario restringido; faster-whisper acepta *initial_prompt* para sesgar el dominio léxico.
- **TTS**: ElevenLabs lidera con clonación de voz y control de estilo, lo cual es clave para dar identidad sonora al agente; Coqui XTTS v2 ofrece clonación local; Piper se limita a voces preentrenadas. La identidad de voz es, por tanto, un factor que puede inclinar la decisión hacia ElevenLabs (nube) o XTTS (local con GPU).

4.4.4. Dimensión 6 — Facilidad de integración

Todos los servicios exponen **API REST**; los de baja latencia añaden **WebSocket** (Deepgram, ElevenLabs) o **SSE** en *streaming* (Gemini, Groq, Ollama), idóneos para el *pipeline* en tiempo real (§5). Para el cliente **Unity (C#)**, los proveedores con SDK o REST documentado (Deepgram, ElevenLabs, Gemini) se integran directamente; los locales se consumen vía el orquestador en Python expuesto por HTTP, evitando dependencias nativas en C#. La madurez de documentación fue mayor en los proveedores comerciales consolidados; los motores locales requieren más trabajo de integración inicial, compensado por su independencia operativa.

5. Arquitectura y Pipeline de Comunicación

El diseño completo del flujo de datos —diagrama de flujo y diagrama de secuencia UML, más la descripción conceptual paso a paso— se documenta en `report/pipeline_diagram.md` (fuente Mermaid) y se reproduce a continuación.

5.0.1. Diagrama de flujo de datos (alto nivel)

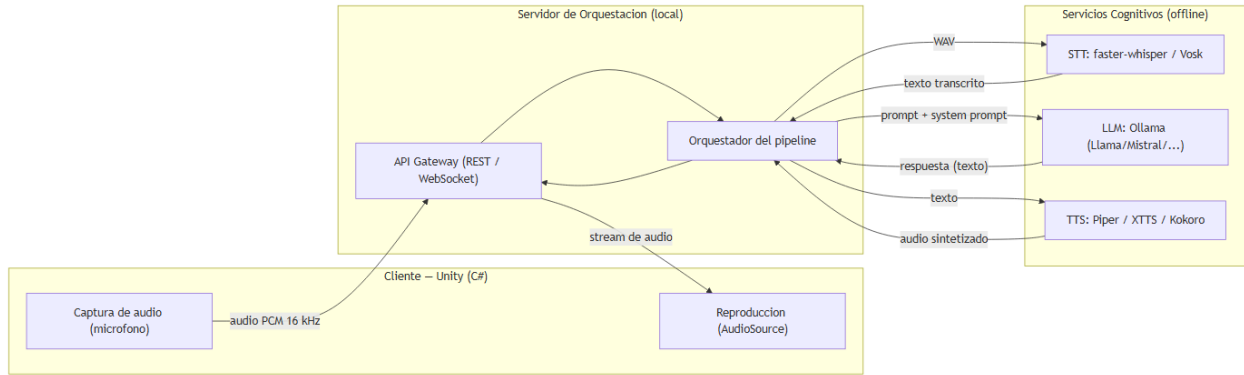


Figura 5: Flujo de datos del pipeline: Unity → STT → LLM → TTS → Unity

5.0.2. Diagrama de secuencia UML (detalle temporal)

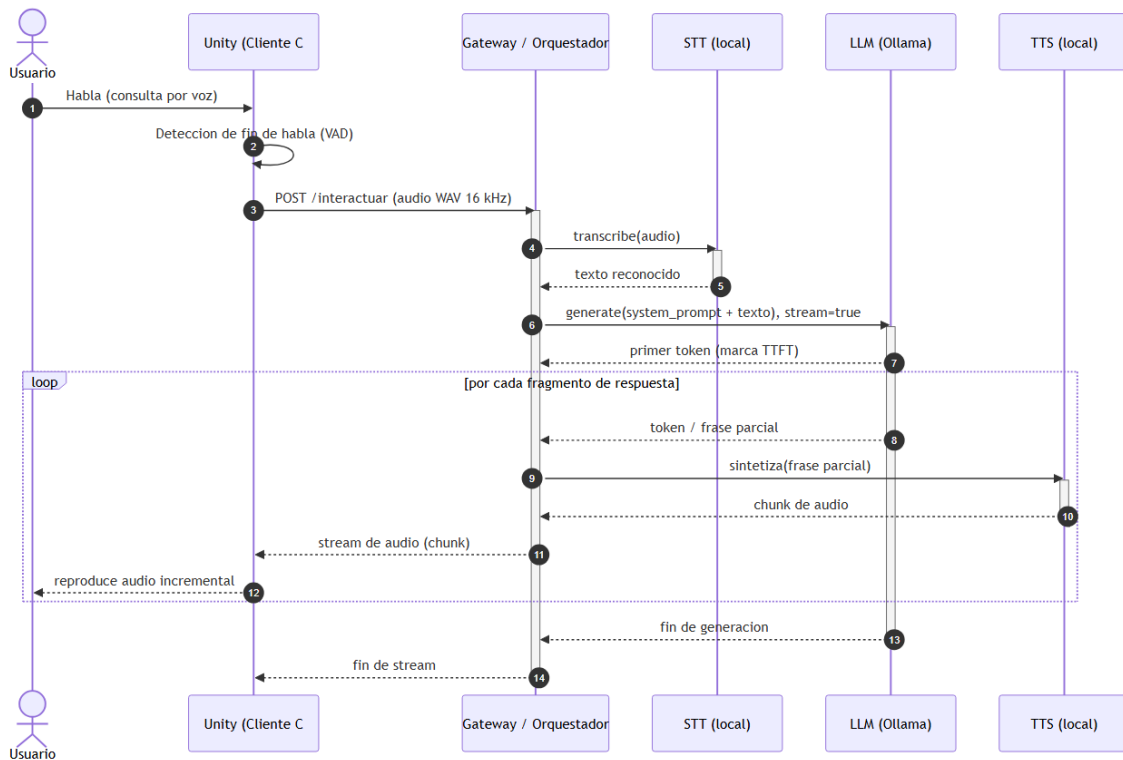


Figura 6: Secuencia UML del pipeline con streaming y síntesis incremental

El *pipeline* integra las tres categorías evaluadas: **Unity (captura de audio) → STT → LLM → TTS → Unity (reproducción)**, orquestado por un servidor local. Las decisiones de diseño orientadas a minimizar la latencia per-

cibida, y respaldadas por los datos de §4 y la literatura [9], son:

1. **Transporte por WebSocket full-dúplex** entre Unity y el orquestador, para *streaming* bidireccional de audio y eventos de turno (*turn-taking*, *barge-in*).
2. **Consumo del LLM en modo *streaming***, aprovechando el TTFT bajo: el primer token dispara de inmediato la síntesis, en lugar de esperar la respuesta completa.
3. **Síntesis incremental**: se envían frases parciales al TTS conforme el LLM las produce, solapando generación y vocalización para reducir la latencia de extremo a extremo por debajo del umbral conversacional.

El banco de pruebas mide de forma aislada las etapas STT, LLM y TTS para fundamentar empíricamente qué combinación minimiza la latencia total del *pipeline*. Sumando los mejores tiempos locales medidos (faster-whisper 0,91 s + TTFT de Phi-3.5 0,03 s + primera frase de Piper $\sim 0,3$ s) se obtiene una latencia percibida de primer audio muy inferior a 1,5 s sin salir de la máquina, lo que valida la viabilidad de un agente privado y responsivo.

6. Recomendaciones según Contexto y Conclusión

Las siguientes recomendaciones contrastan latencia y costo frente a privacidad y personalización, fundamentadas en los datos empíricos de §4.

6.0.1. Escenario A — Privacidad estricta y presupuesto cero (on-premise)

Todo el *stack* local evaluado satisface por diseño la soberanía de datos. Combinación sugerida: **faster-whisper (STT) + Phi-3.5 o Mistral 7B (LLM) + Piper (TTS)**. Operación 100 % offline, huella moderada y latencia razonable sin GPU de gama alta. Ideal para instituciones con datos sensibles (p. ej. trámites estudiantiles). El TTFT de 0,03 s del LLM local es insuperable por cualquier API.

6.0.2. Escenario B — Interactivo en tiempo real, baja latencia (CPU modesta)

Prioriza el TTFT y el RTF mínimo: **Vosk (STT) + Phi-3.5 (LLM) + Piper o eSpeak-NG (TTS)**. Se sacrifica algo de precisión y naturalidad a cambio de la menor latencia de extremo a extremo, adecuado para kioscos o demos en hardware limitado.

6.0.3. Escenario C — Máxima calidad de experiencia (con GPU, local)

Cuando se dispone de GPU y la latencia no es crítica: **faster-whisper (medium/large) + Gemma 2 9B o Qwen 2.5 (LLM) + Coqui XTTS v2 (TTS con clonación de voz)**. Maximiza precisión de transcripción, calidad de razonamiento y naturalidad/identidad de voz del agente, sin que los datos salgan de la institución.

6.0.4. Escenario D — Tiempo real en la nube, mínima latencia (sin GPU propia)

Cuando la prioridad es la latencia más baja posible sin hardware local y se acepta que los datos salgan a un tercero: **Deepgram nova-2 (STT) + gpt-oss-120B o Groq Llama 3.3 70B (LLM, alta velocidad en LPU) + ElevenLabs Flash v2.5 (TTS)**. Aprovecha la infraestructura del proveedor (free tier) para una experiencia muy fluida; gpt-oss-120B aporta calidad *flagship* a 196 tok/s. **Contraparte:** menor privacidad, dependencia de conectividad y de los límites de la capa gratuita.

6.0.5. Escenario E — Máxima calidad de razonamiento propietario (presupuesto disponible)

Si el caso de uso exige el mejor razonamiento comercial y hay presupuesto: **Gemini 2.5 Pro** (con facturación activa, pues su *free tier* es inviable) sobre una base STT/TTS de la nube. Se acepta el costo por token y la dependencia total del proveedor a cambio de la capacidad de la gama más alta. Este escenario solo se justifica cuando la calidad de razonamiento prima sobre costo, latencia y privacidad.

6.0.6. Conclusión

El estudio demuestra que existe un espectro de arquitecturas viables para un agente virtual por voz, desde una solución 100 % local, privada y de costo operativo nulo (Ollama + Whisper local + Piper) hasta una basada en la nube de mínima latencia (gpt-oss-120B / Groq + Deepgram + ElevenLabs) sin hardware propio. La selección óptima no es única: surge del balance que cada escenario exige entre latencia, calidad, costo y privacidad.

El hallazgo central es que la soberanía de datos y la calidad de experiencia se ubican en extremos de un mismo espacio de diseño, pero la brecha es menor de lo que sugiere el mercado: una GPU de gama media iguala o supera a la nube en latencia, y los *flagships* de pesos abiertos (gpt-oss-120B) ofrecen calidad de gama alta sin el costo ineludible del *flagship* propietario (Gemini 2.5 Pro, cuyo *free tier* resultó inviable). Los datos empíricos recolectados con este banco de pruebas constituyen la base objetiva para la decisión arquitectónica y para las fases posteriores del desarrollo del agente.

7. Referencias

- [1] Bhandare, A. et al. (2024). *LLM-Inference-Bench: Inference Benchmarking of Large Language Models on AI Accelerators*. arXiv:2411.00136. <https://arxiv.org/abs/2411.00136>
- [2] *Meta-Metrics and Best Practices for System-Level Inference Performance Benchmarking* (2025). arXiv:2508.10251. <https://arxiv.org/abs/2508.10251>
- [3] *On Evaluating Performance of LLM Inference Serving Systems* (2025). arXiv:2507.09019. <https://arxiv.org/abs/2507.09019>
- [4] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C. & Sutskever, I. (2023). *Robust Speech Recognition via Large-Scale Weak Supervision* (Whisper). OpenAI. <https://cdn.openai.com/papers/whisper.pdf>
- [5] Pratap, V. et al. (2024). *Scaling Speech Technology to 1,000+ Languages* (MMS). *Journal of Machine Learning Research*, 25. <https://jmlr.org/papers/volume25/23-1318/23-1318.pdf>
- [6] Maniati, G. et al. (2022). *SOMOS: The Samsung Open MOS Dataset for the Evaluation of Neural Text-to-Speech Synthesis*. arXiv:2204.03040. <https://arxiv.org/abs/2204.03040>
- [7] Tan, X. et al. (2022). *NaturalSpeech: End-to-End Text to Speech Synthesis with Human-Level Quality*. arXiv:2205.04421. <https://arxiv.org/abs/2205.04421>
- [8] *Benchmarking the Responsiveness of Open-Source Text-to-Speech Systems* (2025). *Computers* (MDPI), 14(10), 406. <https://www.mdpi.com/2073-431X/14/10/406>
- [9] LiveKit; AssemblyAI; Retell AI (2025–2026). *Voice Agent Architecture: STT, LLM y TTS Pipelines* (umbrales de latencia conversacional 300–700 ms). <https://livekit.com/blog/voice-agent-architecture-stt-llm-tts-pipelines-explained>
- [10] Hugging Face (2026). *Best Open-Source / Open-Weight LLM Models to Run Locally*. <https://huggingface.co/blog/daya-shankar/open-source-llm-models-to-run-locally>
- [11] Google (2026). *Gemini API — Pricing* (2.5 Pro \$1.25/\$10, Flash \$0.30/\$2.50, Flash-Lite \$0.10/\$0.40 por 1M tokens). <https://ai.google.dev/gemini-api/docs/pricing>
- [12] Groq (2026). *Groq API Pricing* (gpt-oss-120B \$0.15/\$0.60; Llama 4 Scout; Llama 3.3 70B \$0.59/\$0.79 por 1M tokens; Whisper large-v3 por hora). <https://groq.com/pricing>
- [13] Deepgram (2026). *Pricing* (nova-2 STT por minuto; Aura 2 TTS por carácter) y AssemblyAI *Pricing* (por hora). <https://deepgram.com/pricing> · <https://www.assemblyai.com/pricing>
- [14] ElevenLabs (2026). *Pricing* (multilingual v2 ~ \$0.10/1k car.; Flash v2.5 ~ \$0.05/1k car.). <https://elevenlabs.io/pricing>

8. Anexos

- **Código del banco de pruebas:** benchmarks/, common/, analysis/, tools/.
- **Datos crudos:** results/*.csv y results/*.jsonl.
- **Tablas y figuras generadas:** report/tablas_generadas.md, report/figures/.
- **Diagramas del pipeline:** report/pipeline_diagram.md.
- **Reproducibilidad:** README.md, Dockerfile, docker-compose.yml.